PRECISION UI BDD AUTOMATION FRAMEWORK

(VEEVA SYSTEMS AUTOMATION TESTING ASSIGNMENT)

SUBMITTED BY:

M. ESWARI

CH. SRAVANI

UNDER THE GUIDANCE OF:
VEEVA SYSTEMS

COLLEGE NAME:
S. R. K. R. ENGINEERING COLLEGE

DATE:

12th APRIL, 2026

TABLE OF CONTENTS

- Introduction 2
- Objective 2
- Tech Stack 2
- Framework Architecture 3
- Project Structure 3
- Automation Framework Structure 5
- Implementation Details 5
- Test Scenarios 6
- Test Cases (Pass / Fail) 6
- OOP Concepts 7
- Data Driven Testing 8
- Cross Browser Execution 8
- Reporting & Logging 8
- Execution Results 9
- Maven Execution 10
- Git Integration 11
- Challenges & Solutions 11
- Conclusion & Future Enhancements 11

INTRODUCTION

This document describes the design and implementation of a Hybrid Test Automation Framework developed using Java, Selenium, Cucumber, and Maven. The framework is designed to automate web application testing using Behavior Driven Development (BDD) approach and follows Page Object Model (POM) design pattern. The primary goal of this project is to build a scalable, maintainable, and reusable automation framework that mimics real-world industry standards.

OBJECTIVE

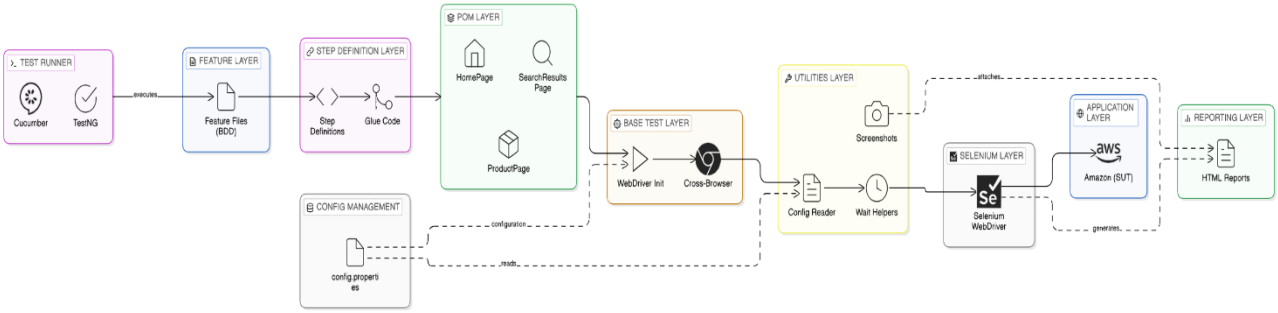
The objective of this project is to:

- Develop a hybrid automation framework using POM and BDD
- Implement reusable and maintainable test scripts
- Support cross-browser execution
- Enable data-driven testing
- Generate detailed execution reports
- Execute tests using Maven commands

TECH STACK

TECHNOLOGY	PURPOSE
Java	Programming Language
Selenium	Web Automation
Cucumber	BDD Framework
Maven	Build Tool
Git	Version Control
Log4j	Logging

FRAMEWORK ARCHITECTURE



PROJECT STRUCTURE

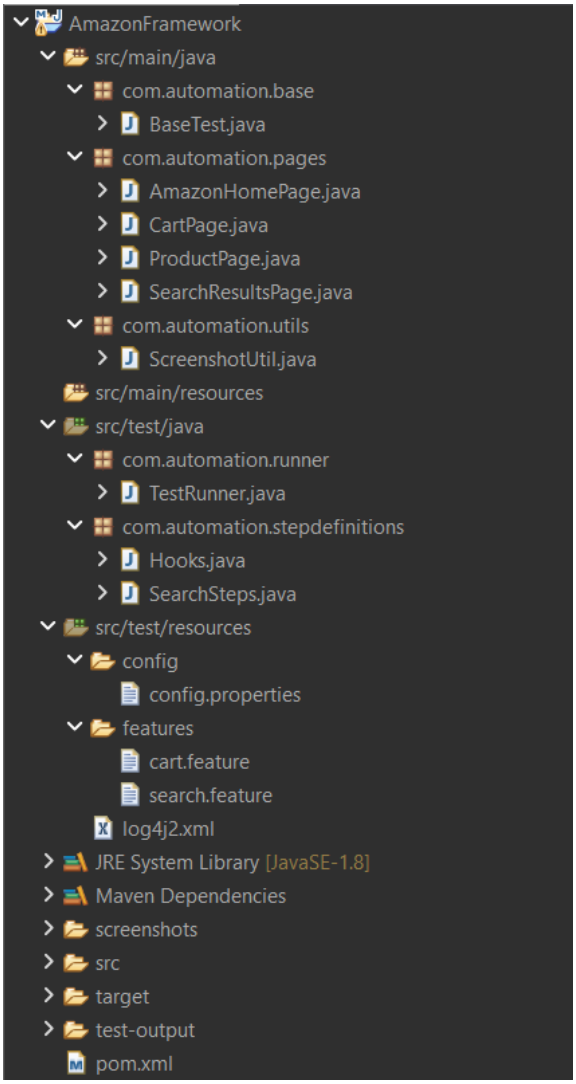


Figure: Project Folder Structure of Automation Framework

The project follows a well-organized Maven-based structure separating test logic, page objects, and configuration files for better maintainability.

The key components of the project are:

- **Base Package (com.automation.base):**
  Contains the BaseTest class which handles WebDriver initialization, browser setup, and teardown operations. It acts as the foundation for all test executions.
- **Page Objects (com.automation.pages):**
  This package includes classes such as AmazonHomePage, SearchResultsPage, ProductPage, and CartPage. These classes store web elements and reusable methods representing different pages of the application, following the Page Object Model (POM) design pattern.
- **Utilities (com.automation.utils):**
  Contains helper classes like ScreenshotUtil used for capturing screenshots during test execution, especially on failures.
- **Step Definitions (com.automation.stepdefinitions):**
  Includes classes like SearchSteps and Hooks. These classes map the Gherkin steps from feature files to Java methods and also handle pre and post execution conditions using Hooks.
- **Runner Class (com.automation.runner):**
  The TestRunner class is responsible for executing the Cucumber feature files and managing test configurations.
- **Feature Files (src/test/resources/features):**
  Contains BDD scenarios written in Gherkin format such as search.feature and cart.feature.
- **Configuration File (config/config.properties):**
  Stores key configuration values like browser type, application URL, and other environment settings.
- **Logging Configuration (log4j2.xml):**
  Used to configure logging mechanism for tracking test execution flow.
- **Additional Folders:**
  - **screenshots:** Stores captured screenshots during test execution
  - **target / test-output:** Contains generated reports and build outputs
  - **pom.xml:** Maven configuration file for managing dependencies and plugins.

AUTOMATION FRAMEWORK STRUCTURE

The framework follows a Hybrid structure combining:

- Page Object Model (POM)
- Behavior Driven Development (BDD)
- Maven build management

Layers in the framework:

- Test Layer – Feature files & Step Definitions
- Page Layer – Page Object classes
- Base Layer – Driver setup
- Utility Layer – Reusable methods
- Config Layer – Properties file

This structure improves maintainability and scalability.

IMPLEMENTATION DETAILS

The framework implementation is designed to ensure reusability, scalability, and efficient test execution.

- **Base Class:**
  The BaseTest class is responsible for initializing the WebDriver and managing browser setup. It ensures proper setup before test execution and handles teardown after execution to close browser instances.
- **Configuration Management:**
  The framework uses a config.properties file to manage configuration values such as browser selection, application URL, and execution settings. This allows easy switching between browsers like Chrome and Edge without modifying the code.
- **Step Definitions & Hooks:**
  Step Definition classes such as SearchSteps implement the actual test logic corresponding to feature files. Hooks class is used to manage preconditions and postconditions like launching and closing the browser.

- **Utilities:**
  Utility classes are implemented to handle common reusable operations:
  - ScreenshotUtil: Captures screenshots on test failure for debugging
  - Wait mechanisms (if implemented): Handles synchronization issues
  - Data handling (if extended): Supports reading external test data
- **Page Object Model Implementation:**
  Each page of the application is represented by a separate class containing web elements and actions. This improves code readability and reduces duplication.
- **Test Execution:**
  Tests are executed using the TestRunner class, which triggers Cucumber scenarios and generates execution reports.

TEST SCENARIOS

The following test scenarios are automated using the Behavior Driven Development (BDD) approach with Cucumber. The scenarios are written in Gherkin language and implemented using step definitions.

Scenario 1: Search for iPhone

Steps:

- User navigates to Amazon homepage
- User enters "iPhone" in the search bar
- User initiates the search
- System displays search results
- Validate that the results are relevant to "iPhone"

Scenario 2: Add Product to Cart

Steps:

- User navigates to Amazon homepage
- User searches for "Mouse"
- System displays search results
- User selects the first product
- User adds the product to the cart
- Validate that the product is successfully added to the cart

TEST CASES (PASS / FAIL)

Test Case ID	Scenario Name	Expected Result	Status
TC_01	Search for iPhone	Relevant products displayed	PASS
TC_02	Add Product to Cart	Product added to cart successfully	PASS

OOPS CONCEPTS

Encapsulation:

Encapsulation is achieved by declaring WebElements as private inside page classes and exposing them through public methods. This hides implementation details and ensures that elements are accessed only through defined actions, improving security and maintainability.

Abstraction:

Abstraction is implemented using base classes and utility classes that contain common functionalities like driver setup, waits, and configuration. It hides complex implementation details and allows test scripts to focus only on high-level actions.

Inheritance:

Inheritance is used where Step Definition classes extend the BaseTest class. This allows reuse of common setup and teardown methods, reducing code duplication and improving consistency across tests.

Polymorphism:

Polymorphism is achieved through method overloading in utility classes, such as different wait methods for various conditions. It allows the same method name to perform different operations based on input parameters, making the framework flexible.

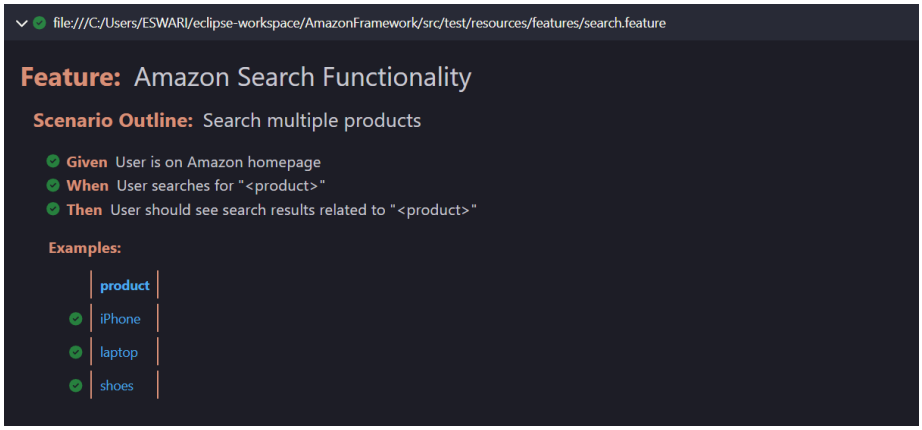
Interface:

Interfaces are used to define browser setup contracts, ensuring a standard way to initialize different browsers. This helps in implementing multiple browser configurations while maintaining a consistent structure.

DATA DRIVEN TESTING

Data Driven Testing (DDT) is a technique where test scripts are executed multiple times using different sets of input data. In this framework, test data is separated from test logic and stored in external sources like Excel or JSON files, making the framework more flexible and maintainable.

Instead of hardcoding values in test scripts, data is passed dynamically using Cucumber Scenario Outline and Examples, allowing the same test scenario to run with multiple data combinations. This reduces redundancy and improves reusability.



CROSS BROWSER EXECUTION

Supports:

- Chrome
- Edge
- Browser selection handled via config.properties
- Uses WebDriverManager for driver setup.

REPORTING & LOGGING

- Generated Cucumber Reports
- Captured screenshots on failure
- Implemented Log4j logging

```
2026-04-14 22:47:37,889 INFO  SearchSteps - Opening Amazon homepage
2026-04-14 22:47:37,913 INFO  SearchSteps - Searching for product: Mouse
2026-04-14 22:49:03,556 INFO  SearchSteps - Selecting product from results
2026-04-14 22:49:40,760 INFO  SearchSteps - Adding product to cart
2026-04-14 22:49:54,291 INFO  SearchSteps - Validating cart details
2026-04-14 22:50:12,996 INFO  SearchSteps - Checking cart count
2026-04-14 22:50:13,509 INFO  SearchSteps - Subtotal value:  ₹299.00
2026-04-14 22:50:13,628 INFO  SearchSteps - Total value:  ₹299.00
2026-04-14 22:50:13,636 INFO  SearchSteps - Test completed successfully
2026-04-14 22:50:37,707 INFO  SearchSteps - Opening Amazon homepage
2026-04-14 22:50:37,710 INFO  SearchSteps - Searching for product: iPhone
2026-04-14 22:52:40,105 INFO  SearchSteps - Validating search results for: iPhone
```

EXECUTION RESULTS

Tests were executed using Maven command: **mvn clean test**

Results:

• All test cases executed successfully

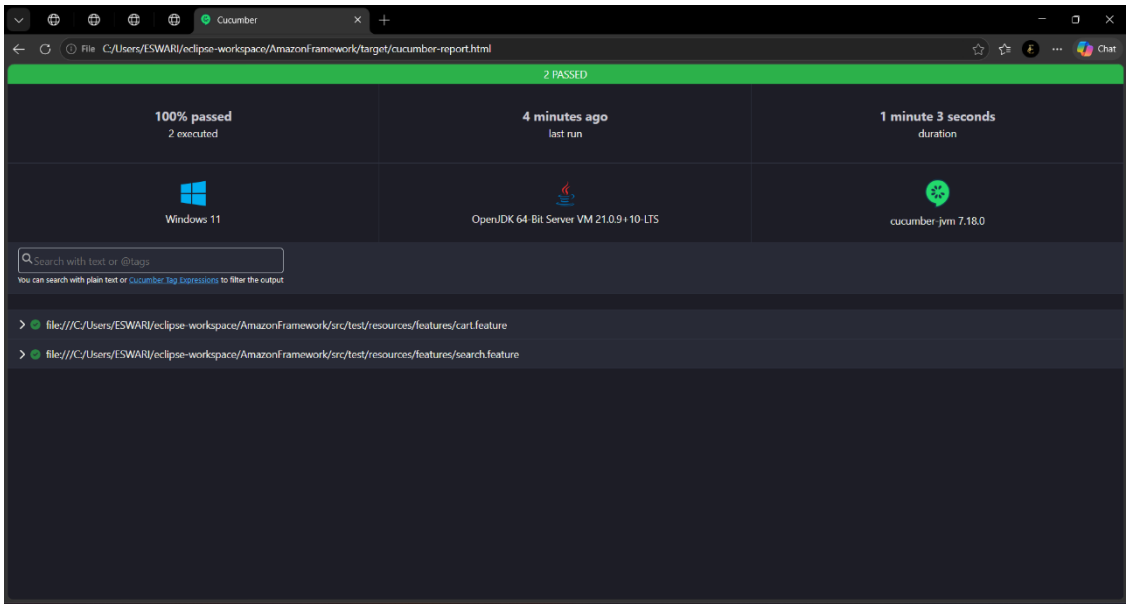
• Reports generated in target folder

• Screenshots captured

• Logs generated in console using Log4j

Report Path:

target/cucumber-report.html



```
file:///C:/Users/ESWARI/eclipse-workspace/AmazonFramework/src/test/resources/features/cart.feature

Feature: Amazon Cart Functionality

  Scenario: Add product to cart

    Given User is on Amazon homepage
    When User searches for "Mouse"
    Then User should see search results related to "Mouse"
    When User selects first product
    Then User adds product to cart
    And Product should be added to cart

file:///C:/Users/ESWARI/eclipse-workspace/AmazonFramework/src/test/resources/features/search.feature

Feature: Amazon Search Functionality

  Scenario: Search for iPhone

    Given User is on Amazon homepage
    When User searches for "iPhone"
    Then User should see search results related to "iPhone"
```

MAVEN EXECUTION

- Run tests using: **mvn clean test**
- Integrated Surefire Plugin
- Supports build lifecycle and report generation.

```
BaseTest ... [TestNG] C:\Users\ESWARI\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.9.v20251105-0741\jre\bin\javaw.exe (14-Apr-2026, 10:47:10 pm elapsed 00:33:2) [pid: 30692]
[RemoteTestNG] detected TestNG version 7.10.2
SLF4J(W): No SLF4J providers were found.
SLF4J(W): Defaulting to no-operation (NOP) logger implementation
SLF4J(W): See https://www.slf4j.org/codes.html#noProviders for further details.

Scenario: Add product to cart         # src/test/resources/features/cart.feature:3
Apr 14, 2026 10:47:27 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find CDP implementation matching 146
Apr 14, 2026 10:47:27 PM org.openqa.selenium.chromium.ChromiumDriver lambda$new$5
WARNING: Unable to find version of CDP to use for 146.0.7680.178. You may need to include a dependency on a specific version of the CDP using something similar to `org.seleniumhq.selenium:selenium-devtool
22:47:37 INFO  SearchSteps - Opening Amazon homepage
  Given User is on Amazon homepage     # com.automation.stepdefinitions.SearchSteps.user_on_homepage()
22:47:37 INFO  SearchSteps - Searching for product: Mouse
  When User searches for "Mouse"       # com.automation.stepdefinitions.SearchSteps.search_product(java.lang.String)
22:49:03 INFO  SearchSteps - Selecting product from results
  And User selects first product       # com.automation.stepdefinitions.SearchSteps.select_product()
22:49:40 INFO  SearchSteps - Adding product to cart
  And User adds product to cart        # com.automation.stepdefinitions.SearchSteps.add_to_cart()
22:49:54 INFO  SearchSteps - Validating cart details
22:50:12 INFO  SearchSteps - Checking cart count
Cart Count: 1
22:50:13 INFO  SearchSteps - Subtotal value:  ₹299.00
22:50:13 INFO  SearchSteps - Total value:  ₹299.00
22:50:13 INFO  SearchSteps - Test completed successfully
  Then Product should be added to cart # com.automation.stepdefinitions.SearchSteps.validate_cart()
```

```
Scenario: Search for iPhone                          # src/test/resources/features/search.feature:3
Apr 09, 2026 8:15:46 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find CDP implementation matching 146
Apr 09, 2026 8:15:46 PM org.openqa.selenium.chromium.ChromiumDriver lambda$new$5
WARNING: Unable to find version of CDP to use for 146.0.7680.178. You may need to include a dependency on a specific version of the CDP using something similar to `org.seleniumhq.selenium:selenium-devtool
20:15:48 [main] INFO  com.automation.stepdefinitions.SearchSteps - Opening Amazon homepage
  Given User is on Amazon homepage                   # com.automation.stepdefinitions.SearchSteps.user_on_homepage()
20:15:48 [main] INFO  com.automation.stepdefinitions.SearchSteps - Searching for product: iPhone
  When User searches for "iPhone"                    # com.automation.stepdefinitions.SearchSteps.search_product(java.lang.String)
20:15:58 [main] INFO  com.automation.stepdefinitions.SearchSteps - Validating search results for: iPhone
  Then User should see search results related to "iPhone" # com.automation.stepdefinitions.SearchSteps.validate_results(java.lang.String)
PASSED: io.cucumber.testng.AbstractTestNGCucumberTests.runScenario("Search for iPhone", "Amazon Search Functionality")
        Runs Cucumber Scenarios
PASSED: io.cucumber.testng.AbstractTestNGCucumberTests.runScenario("Add product to cart", "Amazon Cart Functionality")
        Runs Cucumber Scenarios

===============================================
    Default test
    Tests run: 1, Failures: 0, Skips: 0
===============================================


===============================================
Default suite
Total tests run: 2, Passes: 2, Failures: 0, Skips: 0
===============================================
```

GIT INTEGRATION

- Initialized local repository
- Used meaningful commit messages
- Maintained clean project structure
- Pushed code to remote repository (GitHub)

GitHub Repository Link: Precision-UI-BDD-Automation-Framework

CHALLENGES & SOLUTIONS

1. **Dynamic Elements**

Handled using explicit waits

2. **Reusable Steps**

Created generic step definitions

3. **Cross Browser Issues**

Used WebDriverManager

4. **Data Handling**

Used Scenario Outline

5. **Debugging**

Used logs and screenshots

CONCLUSION & FUTURE ENHANCEMENTS

This project successfully demonstrates the implementation of a real-time automation framework using modern tools and best practices.

The framework is:

- Scalable
- Reusable
- Maintainable
- Industry-ready

It provides a strong foundation for building enterprise-level automation solutions.

Future Enhancements:

• CI/CD integration (Jenkins)

• Parallel execution

• Cloud testing

• Advanced reports